A Dynamic Orchestrator under a Forced Compute Ladder: Off-the-shelf Agent Harnesses against the Fixed Rule

Study `openrouter_dynorch_v1` (ladder: `openrouter_forced_budget_v2`)

September 24, 2026

Abstract

The forced-budget ladder found that, at equal generated tokens $C = kA$, one deep call beats every structured multi-call allocation: deep/16 delivers 0.792 macro accuracy against 0.417 for sample/16 and an oracle pass@16 of 0.580. This study asks whether a dynamic orchestrator beats that fixed rule: an off-the-shelf agent harness that decides for itself whether to spawn sub-agents, revise or synthesise, using the same model (Qwen3.8-27B) under the same total completion budget. Two harnesses, OpenClaude 0.13.0 (H1) and Codex CLI 0.156.1 (H3), and a harness-free control (H0) ran on the ladder's 96 items at rungs $C/A \in \{1, 2, 4, 8, 16, 25\}$. A metering proxy caps every call at the remaining budget and grants one grace of at most 1,024 tokens outside C, as the ladder's uncounted answer call did.

The harnesses did not orchestrate. No harness run spawned a sub-agent: 0 of the 1,152 harness runs in the core, and 0 of all 1,296 harness runs across canary, pilot, main and reruns. OpenClaude made exactly one model call in each of its 576 runs; Codex made a second call in 12 of 576 runs, each time after a planning call, never a delegation.

At $C = 16A$ H1 delivers 0.667 [0.573, 0.760], H3 0.646 [0.552, 0.729], and the same-week, same-host control H0 0.719. In the pre-registered primary family (Holm over four, macro, 96 paired items), H1/16 − deep/16 = −12.5 points [−21.3, −5.2] and H3/16 − deep/16 = −14.6 points [−23.4, −7.1]. The frozen reading rule gives the same verdict for both harnesses: dynamic orchestration loses to the fixed rule. Both are above the sampling baseline: H1/16 − sample/16 = +25.0 and H3/16 − sample/16 = +22.9 points.

The shortfall against deep/16 is not evidence against orchestration, because none took place. The harnesses' one call stops on its own at about 0.59 of the cap at $16A$, while the ladder forced deep/16 to spend all 16,384 tokens. Per realized token, the harness cells lie on or near the deep-call curve.

1 Design

1.1 Arms, cells and budget

Figure 1 shows the cells. Every run of every arm is served by the same metering proxy against the same pinned model, template and sampling as the ladder. The budget unit is completion tokens (reasoning and visible text of every call the proxy serves under the run's key, sub-agents included); the harness is told its budget in one sentence of the task prompt and receives HTTP 402 once it is exhausted. Calls of one run are served one at a time, each granted $\max_tokens = C -$ spent, so the cap is exact by construction: no run in

this report exceeded its cap (0 cap overruns among 1,728 runs). A root call that hits its grant mid-reasoning is closed with one answer continuation of at most 1,024 tokens outside C ("grace"), mirroring the ladder's answer call, which was also outside C.

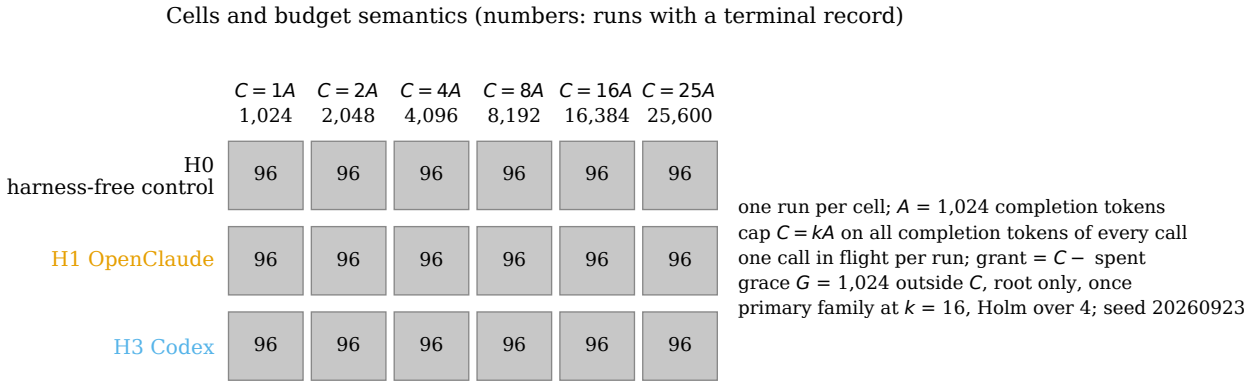


Figure 1: Cells of the core (numbers: runs with a terminal record per arm and rung) and the budget semantics every arm shares.

1.2 Comparison frames

F1 (primary) pairs each harness cell at nominal cap C with the ladder configurations at $C = B \cdot R \cdot A$. The ladder's C counted reasoning only, so F1 is conservative against the harness by one uncounted answer per ladder call: on average 302 tokens for deep/16 and 5,338 for sample/16. F2 (descriptive) puts both sides at realized total completion tokens; the dollar frontier uses charged USD and the standardized uncached price.

1.3 Population and statistics

Every interval is a paired, item-clustered bootstrap over the fixed population of 96 main items (10,000 replicates, seed 20260923), one population per contrast, items kept together across columns. A row exists for every planned cell before a run starts, so no item is ever dropped by the population builder; 0 cells fall below 95.0% completeness. The ladder's own point estimates are reproduced from the merged frame (deep/16 = 0.792 against the sealed 0.792; sample/16 = 0.417 against 0.417); the ladder's intervals are re-drawn per contrast and differ slightly from the sealed report's joint-population intervals.

2 Results

2.1 Headline at $C = 16A$

2.2 Primary family

Anchor first: H0/16 − deep/16 = −7.3 points [−15.5, −0.3] on the same host in the same week, which is the part of any harness shortfall that is not orchestration. Table 2 gives the four pre-registered contrasts with Holm-adjusted p-values over exactly four.

Benchmark	pass@1	vote@16	oracle@16	private/16	best non-deep	deep/16	H0/16	H1/16	H3/16
AIME 25+26	0.333	0.375	0.465	0.500	0.500	0.833	0.792	0.792	0.875
HMMT Feb 25+26	0.217	0.208	0.302	0.292	0.292	0.792	0.750	0.667	0.583
Omni-MATH $\geq$7	0.167	0.292	0.553	0.391	0.391	0.625	0.500	0.458	0.417
GPQA-Diamond	0.625	0.792	0.998	0.917	0.917	0.917	0.833	0.750	0.708
Macro (4)	0.336	0.417	0.580	0.525	0.525	0.792	0.719	0.667	0.646

Table 1: Delivered accuracy at $C = 16A$ per benchmark and macro: the sampling baseline and its oracle, the ladder's revision chain, its best non-deep configuration (`private_single/16`), the deep call, and the three arms.

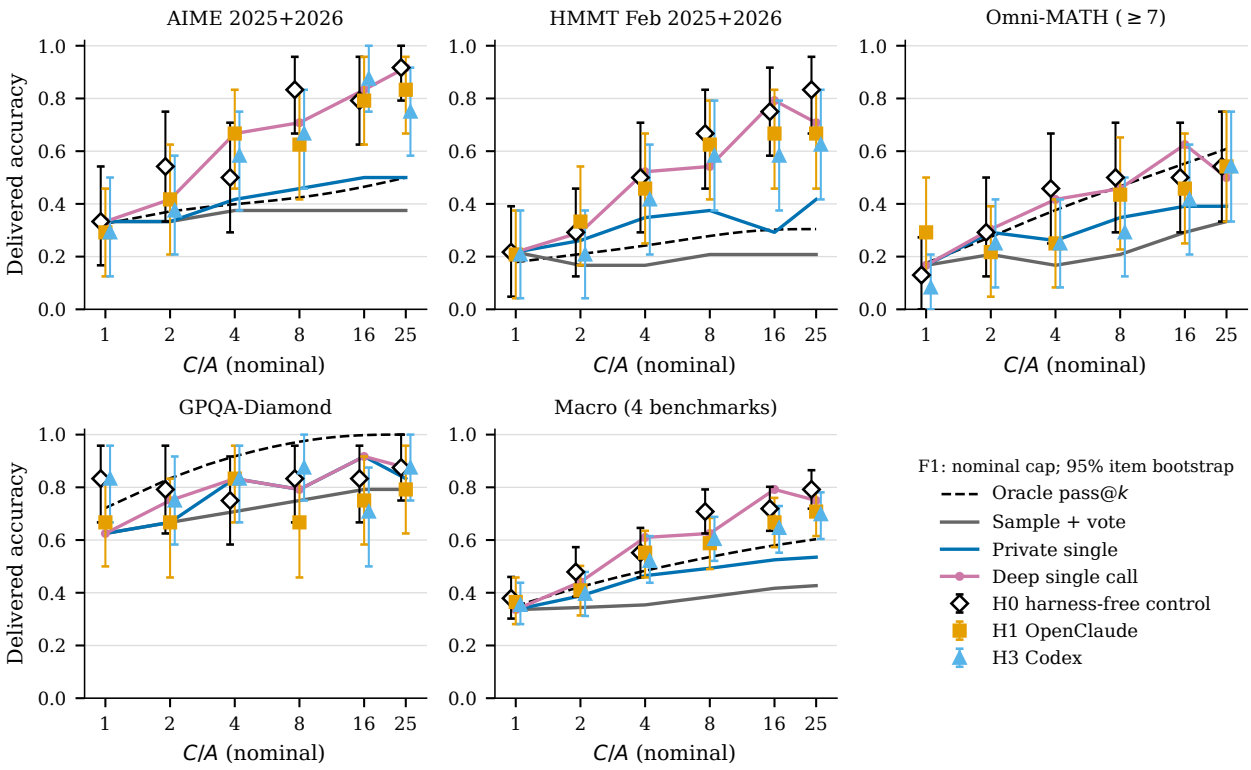


Figure 2: The ladder curves with the harness cells at nominal C (F1). Error bars: 95% paired item bootstrap.

Contrast (macro, 96 items)	estimate (pts)	95% CI (pts)	p	Holm p	reading
`H0/16-deep/16`	−7.3	[−15.5, −0.3]	–	–	loses to fixed rule
`H1/16-deep/16`	−12.5	[−21.3, −5.2]	0.004	0.004	loses to fixed rule
`H1/16-sample/16`	+25.0	[+15.1, +34.7]	0.000	0.000	above baseline
`H3/16-deep/16`	−14.6	[−23.4, −7.1]	0.001	0.003	loses to fixed rule
`H3/16-sample/16`	+22.9	[+13.1, +33.2]	0.000	0.000	above baseline

Table 2: Anchor and the primary family (96 items, macro; studentized item-cluster bootstrap; Holm over four). Reading rule, frozen before collection: CI below 0 against deep/16 means dynamic orchestration loses to the fixed rule at $16A$; CI containing 0 is inconclusive at $n = 96$; CI above 0 means the rule is incomplete.

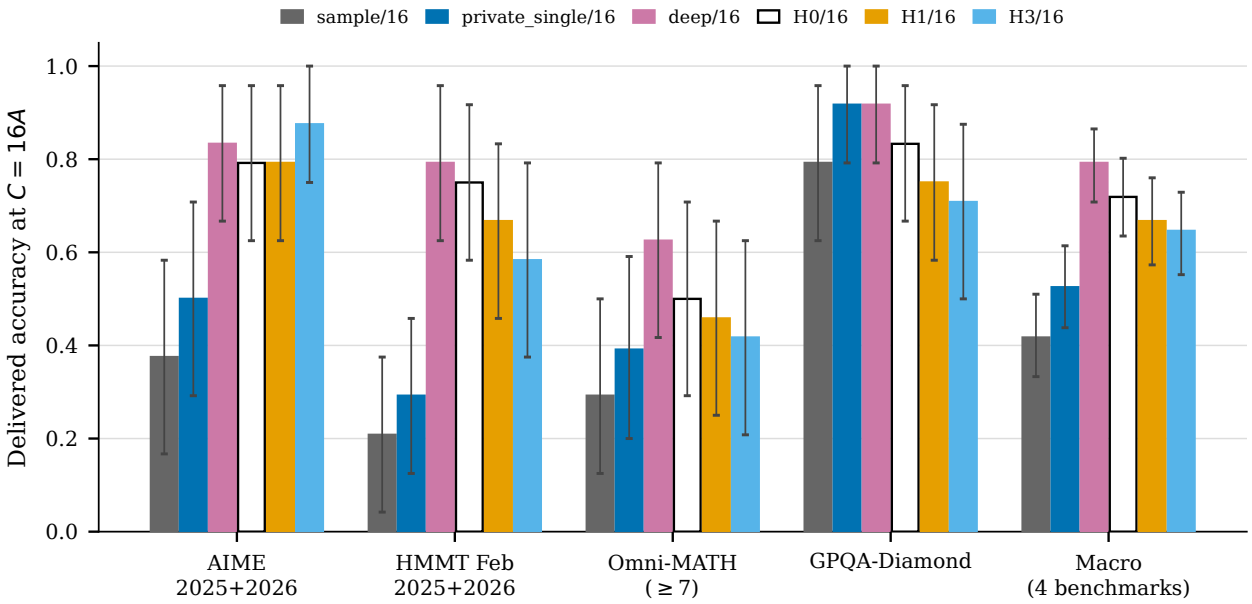


Figure 3: Per benchmark at $C = 16A$ ($n = 24$ each, descriptive).

2.3 Descriptive set, realized frontier and dollars

Slope per doubling of C (macro points): H1 +7.6, H3 +7.7, H0 +8.9, against the ladder's deep +9.6, revision +4.4 and vote +2.1. Against the paired per-item oracle pass@16, H1/16 is +8.8 points [−1.7, +19.2] and H3/16 +7.7 points. The full arm × rung set, with every comparator, is in `tables/rung_table.csv` and `tables/descriptive.csv`.

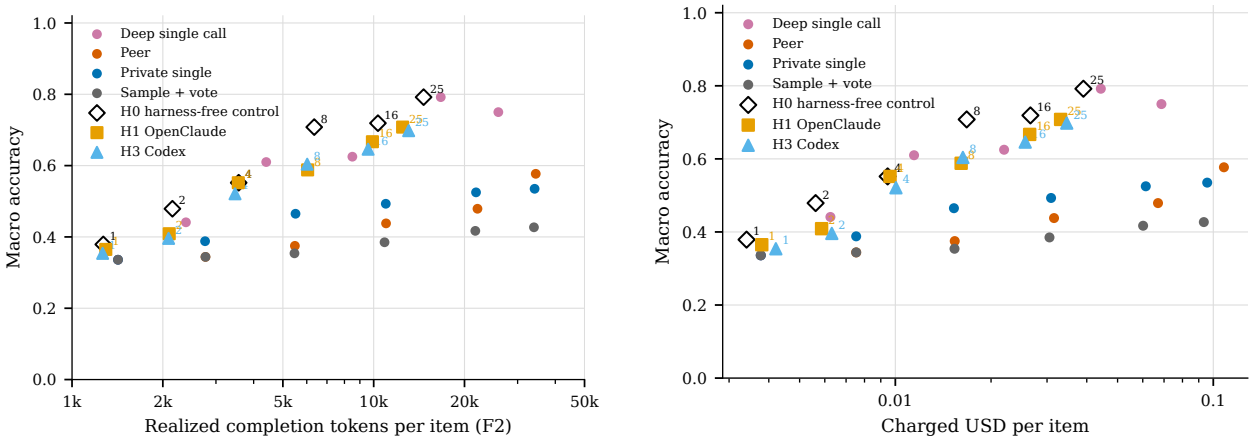


Figure 4: Left: F2, both sides at realized completion tokens per item. Right: charged USD per item. Descriptive frontiers among evaluated configurations.

2.4 Realized budget and harness diagnostics

Scaffolding: the first root call of H1 carries 1,492 more prompt tokens than the H0 call on the same item and rung (SD 5,154, $n = 576$); H3 3,628 (SD 5,540). Spend: $27.42 on the current attempts, $28.13 over every attempt (16 reruns).

configuration	nominal C	realized completion	prompt tokens	USD (charged)	macro accuracy
H0/1	1,024	1,269	1,913	0.0034	0.379
H1/1	1,024	1,294	3,987	0.0038	0.365
H3/1	1,024	1,263	6,254	0.0042	0.354
H0/2	2,048	2,152	2,639	0.0056	0.479
H1/2	2,048	2,100	4,558	0.0059	0.409
H3/2	2,048	2,090	6,740	0.0063	0.396
H0/4	4,096	3,568	3,726	0.0095	0.552
H1/4	4,096	3,558	5,650	0.0096	0.552
H3/4	4,096	3,470	7,948	0.0100	0.521
H0/8	8,192	6,352	5,999	0.0168	0.708
H1/8	8,192	6,040	7,089	0.0161	0.588
H3/8	8,192	6,021	9,471	0.0163	0.604
H0/16	16,384	10,330	7,409	0.0266	0.719
H1/16	16,384	9,904	8,380	0.0265	0.667
H3/16	16,384	9,585	10,702	0.0256	0.646
H0/25	25,600	14,638	8,705	0.0390	0.792
H1/25	25,600	12,472	9,632	0.0331	0.708
H3/25	25,600	13,060	12,966	0.0346	0.698
deep/1	1,024	1,420	1,908	0.0038	0.336
private_single/1	1,024	1,420	1,908	0.0038	0.336
sample/1	1,024	1,420	1,908	0.0038	0.336
deep/2	2,048	2,386	2,932	0.0062	0.441
peer/2/0	2,048	2,769	3,815	0.0075	0.344
peer/2/1	2,048	5,480	10,729	0.0154	0.375
peer/2/3	2,048	10,989	24,258	0.0316	0.438
private_single/2	2,048	2,761	4,686	0.0075	0.388
sample/2	2,048	2,769	3,815	0.0075	0.344
deep/4	4,096	4,403	4,980	0.0115	0.610
peer/4/3	4,096	22,084	66,386	0.0670	0.479
private_single/4	4,096	5,507	10,021	0.0153	0.465
sample/4	4,096	5,463	7,630	0.0154	0.354
peer/5/4	5,120	34,455	121,374	0.1080	0.577
deep/8	8,192	8,498	9,075	0.0220	0.625
private_single/8	8,192	10,968	20,749	0.0309	0.493
sample/8	8,192	10,864	15,260	0.0305	0.385
deep/16	16,384	16,686	17,267	0.0443	0.792
private_single/16	16,384	21,841	42,102	0.0613	0.525
sample/16	16,384	21,722	30,521	0.0602	0.417
deep/25	25,600	25,894	26,483	0.0687	0.750
private_single/25	25,600	34,155	66,282	0.0957	0.535
sample/25	25,600	33,985	47,689	0.0934	0.427

Table 3: Realized tokens and dollars per item for the comparators and the harness cells.

cell	C	mean	median	realized share	grace	cap-bound	terminal
H0/1	1,024	981	1,024	0.958	290	0.885	0.000
H0/2	2,048	1,864	2,048	0.910	288	0.812	0.000
H0/4	4,096	3,336	4,096	0.814	232	0.688	0.000
H0/8	8,192	6,124	8,192	0.748	228	0.625	0.000
H0/16	16,384	10,180	11,232	0.621	150	0.406	0.000
H0/25	25,600	14,525	14,209	0.567	113	0.312	0.000
H1/1	1,024	963	1,024	0.941	331	0.865	0.000
H1/2	2,048	1,794	2,048	0.876	312	0.750	0.000
H1/4	4,096	3,301	4,096	0.806	257	0.656	0.000
H1/8	8,192	5,794	8,192	0.707	272	0.521	0.000
H1/16	16,384	9,730	10,733	0.594	174	0.344	0.000
H1/25	25,600	12,346	10,760	0.482	126	0.271	0.000
H3/1	1,024	961	1,024	0.938	302	0.865	0.000
H3/2	2,048	1,797	2,048	0.877	294	0.750	0.000
H3/4	4,096	3,212	4,096	0.784	258	0.677	0.000
H3/8	8,192	5,807	8,192	0.709	214	0.542	0.000
H3/16	16,384	9,441	10,189	0.576	144	0.344	0.000
H3/25	25,600	12,934	11,094	0.505	126	0.292	0.000

Table 4: Nominal cap beside realized completion (mean, median), realized share, mean grace, share of runs with a cap-bound call and share closed by the terminal answer.

cell	calls	root	solver	spawns	spawn share	orch. share	abstain	parse fail	wall s
H0/1	1.0	1.0	0.0	0.00	0.000	1.000	0.010	0.000	19
H0/2	1.0	1.0	0.0	0.00	0.000	1.000	0.000	0.000	34
H0/4	1.0	1.0	0.0	0.00	0.000	1.000	0.000	0.000	56
H0/8	1.0	1.0	0.0	0.00	0.000	1.000	0.000	0.000	95
H0/16	1.0	1.0	0.0	0.00	0.000	1.000	0.000	0.000	150
H0/25	1.0	1.0	0.0	0.00	0.000	1.000	0.000	0.000	217
H1/1	1.0	1.0	0.0	0.00	0.000	1.000	0.052	0.000	21
H1/2	1.0	1.0	0.0	0.00	0.000	1.000	0.062	0.000	34
H1/4	1.0	1.0	0.0	0.00	0.000	1.000	0.021	0.000	53
H1/8	1.0	1.0	0.0	0.00	0.000	1.000	0.062	0.000	88
H1/16	1.0	1.0	0.0	0.00	0.000	1.000	0.042	0.000	140
H1/25	1.0	1.0	0.0	0.00	0.000	1.000	0.031	0.000	170
H3/1	1.0	1.0	0.0	0.00	0.000	1.000	0.062	0.000	20
H3/2	1.0	1.0	0.0	0.00	0.000	1.000	0.094	0.000	33
H3/4	1.0	1.0	0.0	0.00	0.000	1.000	0.000	0.000	52
H3/8	1.0	1.0	0.0	0.00	0.000	1.000	0.031	0.000	88
H3/16	1.0	1.0	0.0	0.00	0.000	1.000	0.000	0.000	133
H3/25	1.1	1.1	0.0	0.00	0.000	1.000	0.021	0.000	185

Table 5: Harness diagnostics per cell: calls by conversation class, spawns, orchestrator token share, abstention (no valid <final>), tool-call parse failures and wall time.

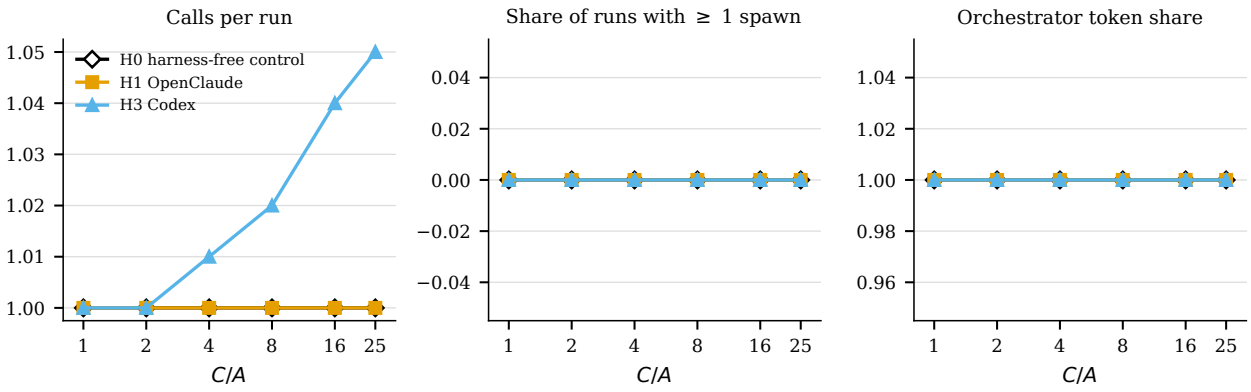


Figure 5: Calls per run, spawn share and orchestrator token share against C/A.

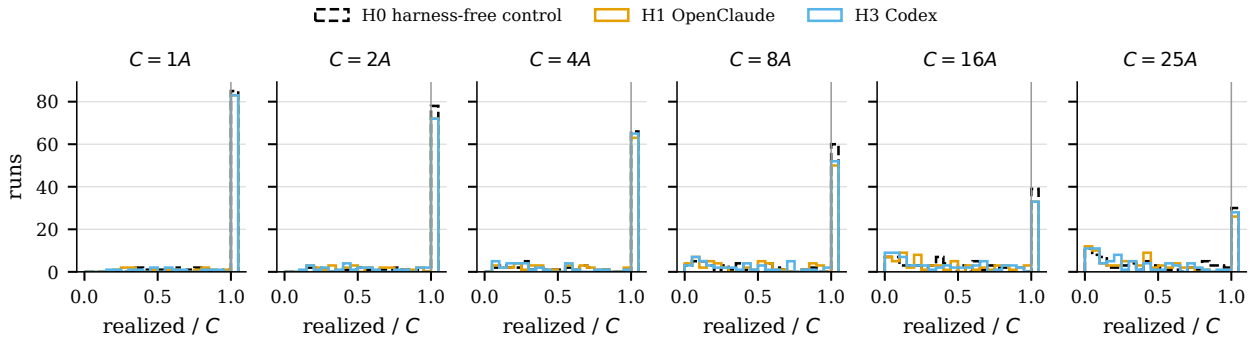


Figure 6: Realized share of the cap per run, by rung and arm.

2.5 Provider sensitivity

58 of the ladder's items were generated on Mancer and 38 on Parasail; 100.0% of the harness cells ran on the same provider as their ladder rows (96 items).

contrast	Mancer items	Parasail items	same provider only
`H0/16-deep/16`	−6.9 [−17.0, +1.9] (n=58)	−8.3 [−19.2, +2.2] (n=38)	−7.3 [−14.6, +0.0] (n=96)
`H1/16-deep/16`	−15.4 [−27.4, −4.8] (n=58)	−8.1 [−18.3, +0.0] (n=38)	−12.5 [−20.8, −5.2] (n=96)
`H1/16-sample/16`	+24.9 [+13.0, +36.9] (n=58)	+25.6 [+10.0, +41.1] (n=38)	+25.0 [+15.6, +34.4] (n=96)
`H3/16-deep/16`	−11.9 [−22.1, −1.7] (n=58)	−18.9 [−32.2, −8.1] (n=38)	−14.6 [−22.9, −7.3] (n=96)
`H3/16-sample/16`	+28.3 [+16.4, +40.4] (n=58)	+14.7 [−0.6, +30.0] (n=38)	+22.9 [+13.5, +32.3] (n=96)

Table 6: Every primary contrast restricted to the Mancer items, the Parasail items, and the items whose harness run and ladder rows share a provider.

2.6 Repeat agreement

Extension E1 re-ran every H1 and H3 core cell once more (1,152 runs, $18.16; per-cell numbers in `tables/repeat_agreement.csv`). The second repeat behaved like the first: 1 of 1,152 runs spawned a sub-agent and 12 made more than one call. Per item, the two repeats agree on correctness in 75.8% to 83.3% of items (Cohen's κ 0.49 to 0.62). In 11 of 12 cells the interval for repeat 2 − repeat 1 includes zero (Figure 8); the exception is H3/1 (−8.3 points).

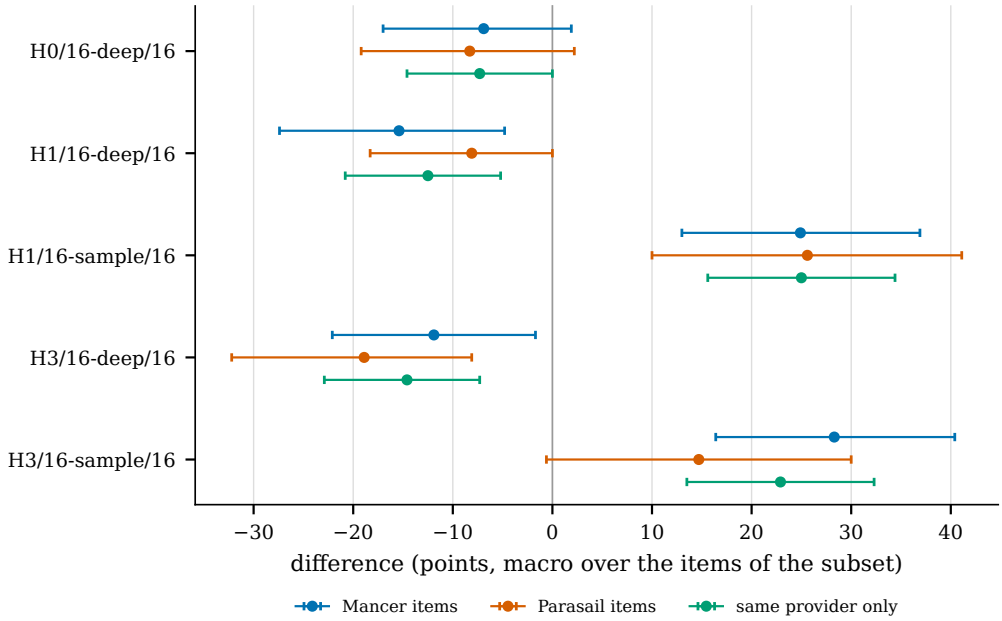


Figure 7: Provider-stratified estimates of the anchor and the primary family.

The primary result replicates. Averaging the two repeats at $16A$ gives H1 − deep/16 = −12.0 points [−19.4, −5.4] and H3 − deep/16 = −11.5 [−18.8, −5.1] (descriptive, paired item bootstrap). Repeat 2 alone scored 0.677 (H1/16) and 0.708 (H3/16). Even the harness's own pass@2 at $16A$, correct in either repeat, which spends twice the budget and needs an oracle to pick, is 0.771 (H1) and 0.760 (H3), below deep/16 at 0.792.

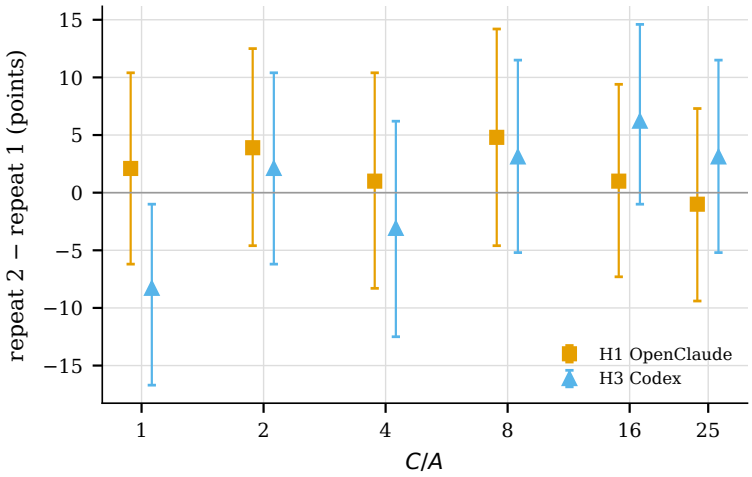


Figure 8: Repeat 2 minus repeat 1 per cell with the empirical run-to-run interval, when E1 ran.

3 Interpretation

The orchestrators did not orchestrate. Across all 1,728 main runs, 0 spawned a sub-agent, at any budget. OpenClaude made exactly one model call in each of its 576 runs. Codex made one call in 564 runs and a second in 12 (2.1%). In every one of those, the first call had invoked `update_plan`, Codex's planning tool,

and the second call wrote the answer; none delegated. Both harnesses exposed delegation tools (`Agent` in OpenClaude; `spawn_agent` and its companions in Codex), and the task prompt stated the budget. The model spent the budget thinking in one call. Behaviourally, the "dynamic orchestrator" in this study is one natural-length call inside the harness's scaffolding.

At the nominal budget the fixed rule wins. H1/16 − deep/16 = −12.5 points [−21.3, −5.2], $p_{\text{Holm}} = 0.004$; H3/16 − deep/16 = −14.6 points [−23.4, −7.1], $p_{\text{Holm}} = 0.003$. The anchor splits each gap into two parts (Table 7). H0/16 − deep/16 = −7.3 points [−15.5, −0.3] is the cost of a natural stop against a forced, exact spend; it is common to both harnesses. H/16 − H0/16 is the scaffolding part: −5.2 points [−14.0, +3.1] for H1 and −7.3 points [−16.4, +1.2] for H3. The harness adds about 1.5k (H1) and 3.6k (H3) prompt tokens of tool schemas and wrappers that no run used, and the point estimates say this costs accuracy. Both intervals include zero, so at 96 items the size of the scaffolding cost is not pinned down. The decomposition is descriptive, with no p-values: it was not in the primary family.

harness	total H/16 − deep/16	natural stop H0/16 − deep/16	scaffolding H/16 − H0/16	prompt tokens added
H1	−12.5 [−21.3, −5.2]	−7.3 [−15.5, −0.3]	−5.2 [−14.0, +3.1]	1,492
H3	−14.6 [−23.4, −7.1]	−7.3 [−15.5, −0.3]	−7.3 [−16.4, +1.2]	3,628

Table 7: The gap to deep/16 at $C = 16A$ split by the anchor (macro points, 95% studentized item-cluster bootstrap CIs, 96 paired items). The two parts add up to the total. The first column is the primary contrast; the scaffolding column is descriptive. Prompt tokens added: the first root call's prompt minus the H0 call's prompt on the same item and rung, mean over all rungs.

Why the natural call loses at nominal C: it stops early. At $16A$ the mean realized share of the cap is 0.59 for H1, 0.58 for H3 and 0.62 for H0 (Table 4); only 34.4% of H1/16 runs and 34.4% of H3/16 runs reached the cap. The ladder forced deep/16 to spend exactly 16,384 reasoning tokens. Per realized token (F2, Figure 4, left), the harness points lie on or near the deep-call curve. Table 8 makes this exact: each cell against the deep-call accuracy interpolated at the same realized completion tokens. For the 10 harness cells the difference runs from −4.1 to +0.4 points (mean −1.2 for H1, −1.7 for H3), and 0 of the 10 intervals exclude zero. H0 sits above the curve in all 5 cells, by +0.7 to +8.3 points; the interval excludes zero at `H0/8`.

The implication is narrow. The rule's advantage at the nominal budget comes from making the model spend its whole budget. The ladder did that by forcing decoding, and no off-the-shelf harness offers it. It is not evidence that orchestration decisions are worse: no orchestration decision was made.

Against the naive baseline and the oracle. Both harnesses beat sample/16 with vote by a wide margin: H1/16 − sample/16 = +25.0 points [+15.1, +34.7], $p_{\text{Holm}} < 0.001$; H3/16 − sample/16 = +22.9 points [+13.1, +33.2], $p_{\text{Holm}} < 0.001$. Both also exceed the paired per-item oracle pass@16 (0.580): H1/16 by +8.8 points [−1.7, +19.2] and H3/16 by +7.7 points [−2.1, +17.2]. Those two intervals include zero. In point estimate, one unforced call at $16A$ does better than a perfect selector over the samples of sample/16 would.

Scope. One model (Qwen3.8-27B); 96 items on 4 hard math and science benchmarks; orchestration-only tools (no code execution, no search); one realisation per cell. Orchestration itself was not exercised, so this does not show that orchestration is useless. It shows that these two harnesses, as shipped, did not choose to orchestrate this model on these tasks. Section 4 tests whether forcing the root to stop early makes it delegate.

cell	realized tokens	accuracy	deep at same tokens	difference (pts) [95% CI]
H0/2	2,152	0.479	0.420	+6.3 [−1.1, +13.8]
H0/4	3,568	0.552	0.552	+0.7 [−6.5, +8.0]
H0/8	6,352	0.708	0.618	+8.3 [+2.2, +15.0]
H0/16	10,330	0.719	0.673	+4.6 [−3.5, +12.6]
H0/25	14,638	0.792	0.760	+3.2 [−2.8, +9.4]
H1/2	2,100	0.409	0.415	−0.6 [−9.1, +8.0]
H1/4	3,558	0.552	0.551	−0.6 [−9.1, +8.1]
H1/8	6,040	0.588	0.617	−4.1 [−11.9, +3.5]
H1/16	9,904	0.667	0.663	+0.4 [−7.3, +7.5]
H1/25	12,472	0.708	0.720	−1.1 [−8.1, +5.0]
H3/2	2,090	0.396	0.414	−1.6 [−8.4, +5.0]
H3/4	3,470	0.521	0.544	−2.0 [−10.1, +6.3]
H3/8	6,021	0.604	0.617	−0.9 [−8.3, +6.3]
H3/16	9,585	0.646	0.655	−0.9 [−9.5, +7.4]
H3/25	13,060	0.698	0.731	−3.3 [−11.6, +4.0]

Table 8: Each cell against the deep call at the same realized completion tokens per item (F2, macro, descriptive). The deep accuracy is interpolated log-linearly in realized tokens between the two adjacent deep rungs. The difference and its 95% CI are one paired linear contrast (cell minus the weighted pair of deep columns) over the items known in all three columns, so it can differ by a point from the difference of the two printed accuracies. The rung-1 cells realized fewer tokens than deep/1 (1,420) and are left out.

4 Extension E0: capping the orchestrator's own calls

In the core, both harnesses almost always answered in one long call and never delegated. E0 takes away the single long call. Each root call is granted at most `max(A, C/4)` tokens: 1,024 at $4A$ and 4,096 at $16A$. A root call that reaches this ceiling is closed with an answer continuation charged inside C. The harness can then accept that answer, or keep going with the rest of the budget: think again, or spawn a sub-agent. The same 96 items were run for H1c and H3c at rungs 4 and 16, one realisation per cell (384 runs, $3.97). Every contrast below is a descriptive paired item bootstrap, and E0 carries no p-values.

cell	accuracy	− uncapped	− deep/k	− deep at ceiling	spawn runs	ceiling close, then end
H1c/4	0.323	−22.9	−28.5	−1.1 (deep/1)	11.5%	86.9%
H1c/16	0.490	−17.7	−30.2	−11.7 (deep/4)	19.8%	72.1%
H3c/4	0.302	−21.9	−30.5	−3.1 (deep/1)	6.2%	72.0%
H3c/16	0.556	−9.4	−23.1	−5.3 (deep/4)	5.2%	66.2%

Table 9: E0 cells: macro accuracy and paired differences in points. "Deep at ceiling" is one forced deep call with the root ceiling as its budget. The last column is the share of ceiling-closed runs in which the harness made no further call.

The cap made the harnesses delegate, sometimes. 41 of 384 E0 runs spawned a sub-agent, against none of the core harness runs. The share was highest for H1c/16 (19.8%) and lowest for H3c/16 (5.2%).

Mostly they accepted the capped answer and stopped. The root reached its ceiling in 67.7% (H3c/16) to 87.5% (H1c/4) of runs. In 66.2% to 86.9% of those, the harness made no further call and delivered the

answer written under the ceiling. The unspent budget was simply left unused: realised share of the cap is 0.29 to 0.41. So a capped cell at rung k behaves much like one deep call at the ceiling. H1c/4 is -1.1 points [-8.7, +6.2] from deep/1, and H3c/16 is -5.3 [-15.7, +5.1] from deep/4.

Capping cost accuracy. Every capped cell is below its uncapped cell in point estimate, and three of the four intervals exclude zero. H1c/16 is lower by 17.7 points [-27.8, -7.9] and H3c/16 by 9.4 [-19.8, +1.0]. Against deep/16 the gap widens to -30.2 and -23.1 points. Runs whose root never hit the ceiling were right 92.9% (H1c/16) and 100.0% (H3c/16) of the time; ceiling-closed runs were right 30.9% and 34.4%. That split is selection by difficulty, not an effect of capping. Runs that spawned did no better (H1c/16: 15.8% with a spawn, 57.1% without). This is also selection, and the counts are small.

Reading. Forcing the orchestrator off its first call did not produce useful orchestration. When the harnesses were pushed to split the work, they mostly turned the budget they had into a shorter single answer, and the delegation they did perform did not recover the lost depth. E0 strengthens the core reading: for this model at these budgets, the rule "one deep call, forced to the budget" is not beaten by dynamic orchestration, capped or uncapped.

5 Extension E2: the same harnesses without the spawn tool

No core run delegated, yet both harnesses fell below the harness-free call H0 at $16A$. E2 asks whether merely offering the delegation tool is part of that cost. It re-ran both harnesses at rungs 4 and 16 with spawning disabled: OpenClaude with no tools at all, and Codex without its `multi_agent_v1` tools, keeping only `update_plan`. That is 384 runs for $6.45; 0 spawned, as designed. Without the spawn tool the first root prompt is shorter: 6,363 against 8,380 tokens (OpenClaude, $16A$), and 7,995 against 10,243 (Codex).

cell	accuracy	− spawn offered (both repeats)	− H0/k	− deep/k
H1n/4	0.534	-2.6 [-11.1, +5.8]	-2.3 [-11.5, +7.3]	-8.6 [-19.2, +2.1]
H1n/16	0.740	+6.8 [+0.2, +13.4]	+2.1 [-6.0, +10.4]	-5.2 [-12.8, +2.1]
H3n/4	0.594	+8.9 [+2.0, +16.7]	+4.2 [-3.7, +12.4]	-1.1 [-10.4, +7.4]
H3n/16	0.740	+6.2 [-1.9, +14.6]	+2.1 [-5.7, +9.9]	-5.2 [-12.4, +0.5]

Table 10: E2 cells: macro accuracy and paired differences in points (95% item bootstrap, descriptive). "Spawn offered" is the mean of the core cell and its E1 repeat.

Removing the unused tool helped, mostly. Against the mean of both spawn-offered repeats, three of the four cells gained +6.8 (H1n/16), +6.2 (H3n/16) and +8.9 (H3n/4) points. The fourth, H1n/4, moved -2.6. Of the 4 intervals, 2 include zero. Every spawn-disabled cell is within a few points of the harness-free call H0/k, and every one of those intervals includes zero. At $16A$ both lie 5.2 points below deep/16, and those intervals include zero as well.

Reading. The core's gap between the harnesses and H0 (Table 7) looks like the cost of offering a delegation tool that is never used: a longer system prompt and multi-agent instructions, with nothing gained. Without it,

a harness performs like one unforced call. The remaining gap to deep/k is the natural-stop gap of H0: the model stops before spending the budget, which the ladder's forced call does not allow. These are descriptive contrasts from one realisation per E2 cell, run later than the core, so read them as consistent with that account, not as proof of it.

6 What to do with this

For this model on these tasks, spend the budget on one deep call and force it to use the whole budget; neither harness adds anything over that, and each adds unused prompt overhead. If a harness is used for other reasons, expect it to behave like one unforced call that stops at about 0.59 of the cap. If the task does not need delegation, do not offer the tool: in E2 removing it recovered most of the gap to the harness-free call. The rule would change with a harness that actually delegates, with tools that add information a single call cannot produce (code execution, search), or with a model that spends a long budget without being forced.

7 Cost re-fit

Re-fitted from the measured runs (Parasail prices, 25.0% contingency), the core would cost $26.12; per-cell figures are in `tables/forecast_refit.csv`.

A Run facts and deviations

Spend. The current attempts of the core cost $27.42. The ledger charges every main attempt together $28.13: $27.44 settled by the provider plus $0.69 charged at the full reservation for the 16 calls the provider cut mid-stream, which the provider billed at close to nothing (contract §10.39). The 16 attempts that reruns replaced cost $0.72. The canary cost $0.50 over both executions (the superseded first one $0.25), and the pilot $2.65. The study total, canary and pilot included and E0 excluded, is $30.59 settled, at most $31.28 with the conservative charges. The OpenRouter key agreed with the ledger at every gate.

Gates. Gate 1 (canary) and Gate 2 (pilot) advanced with owner-waived delegation criteria (contract §10.37). The canary's $16A$ probe had 0 of 4 runs spawning, which trips the pre-registered stop-and-ask. At Gate 2, 0 of 32 H1 runs and 0 of 32 H3 runs at $16A$ spawned. The owner's decision, recorded in `run_decisions.jsonl`: "Run the core as planned", because "non-delegation is the finding; waive the pilot's delegation threshold and the live Codex tool-loop check (verified offline at Gate 0); the capped variant stays first in line after the core". The waived criteria stay recorded as failed, and nothing else was waivable: money, accounting, grants, provider pinning, answers and forecast criteria stayed binding. The final Gate 4 passed every criterion.

Defects found and fixed during the run.

- **The grace offered tools (§10.28).** In the first canary the answer continuation after a cap-bound call still carried the tool schemas, so some harness runs spent their one grace on a tool call and had no

budget left to answer. Every spawn recorded anywhere in the study (1 in all) was issued inside such a grace. The grace now carries no tools, as the plan said; the 10 canary cells were superseded and re-run (§10.35).

- **The upstream-failure latch (§10.38).** The provider cut 16 main-stage calls mid-stream (12 error envelopes, 4 ambiguous timeouts). H0 recorded these as transport failures and re-ran them (8 reruns). Both harnesses instead retried the failed call into the proxy's no-thinking terminal answer, which was scored as a model outcome. The first upstream failure now ends the run as an infrastructure rerun, and the 8 main cells completed under the defect (5 H1, 3 H3) were invalidated and re-executed.
- **Reconciliation rules (§10.36, §10.39, §10.41).** The key's usage lags the ledger, so the dollar check now waits for it to converge; it compares the key with the ledger's bounds (settled, settled plus conservative charges) rather than the upper figure; and a failed call charged at its full grant is checked as an upper bound. No recorded dollar changed.
- **Gate 4's grader re-run (§10.40).** The first Gate 4 counted 26 mismatches, all the same `INVALID_OUTPUT` verdict both times: a well-formed final answer the grader cannot parse, scored 0. The check compared the raw verdict instead of the scored value. It now uses the scoring rule; stored scores were already right and did not change.

The first main launch. It was submitted without the account auto-reload flag, so its active ceiling was $49.84 instead of the approved $95.00. It was drained after 5.3 minutes and resubmitted with the flag; its 50 finished cells were kept.

None of these changed a pre-registered scientific rule. Each made the code match the plan or fixed a check.